IT2080 IT Project

Year 2, Semester 2, 2025

Activity 04 - Database Design & Development Activity

Mobile, Electronics and Service

Group Details

Campus: Malabe Campus

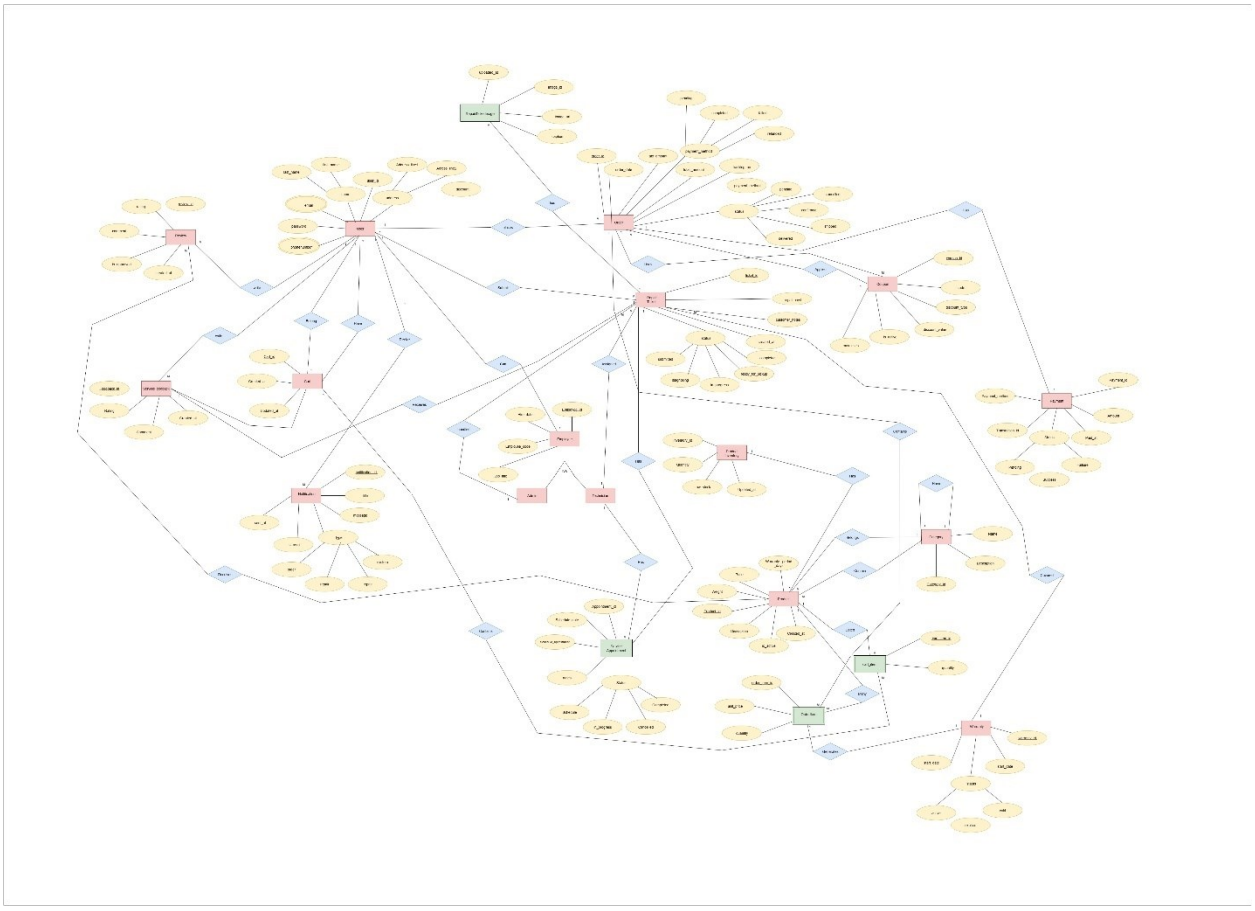
ITP Group Number: ITP25_WE_B1_215

IT Number	Student Name	Email	Contact Number
IT23571884	Meshech Johnathan P R	IT23571884@my.sliit.lk	0716610474
IT23313248	K. Mathuran	IT23313248@my.sliit.lk	0760837551
IT23542242	Thushanya J	IT23542242@my.sliit.lk	0752414453
IT23540194	A.A.R.A.M Shakna	IT23540194@my.sliit.lk	0777076743
IT23614758	Thayaparan Danuskanth	IT23614758@my.sliit.lk	0752695486

Contents

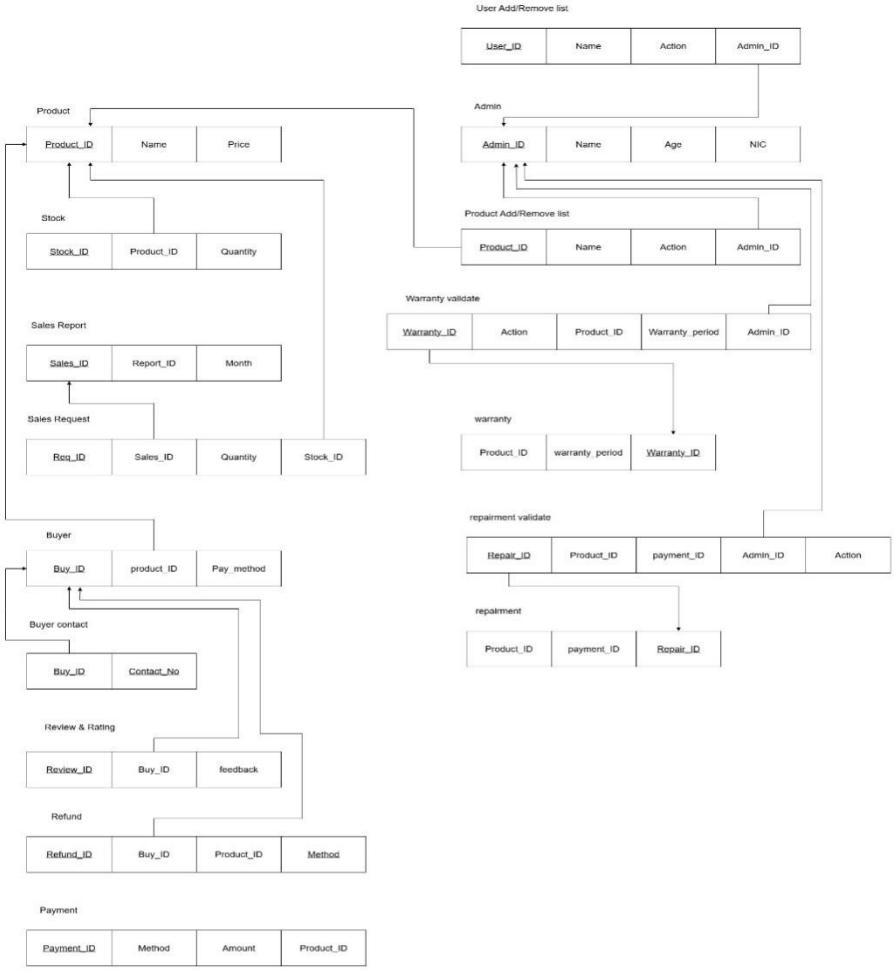
1.ER diagram 3
2. Relational Model 4
3. Database Implementation 5

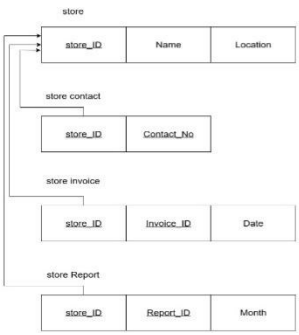
1.ER diagram



Link – ER-Diagram

2. Relational Model





Link – Relational Model

3.Database Implementation

User Model

```
import mongoose from 'mongoose';
import bcrypt from 'bcryptjs';

const userSchema = mongoose.Schema(
  {
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    profilePicture: {
      type: String,
      //default: '/uploads/default-avatar.png',
    },
    role: { type: String, required: true, default: 'user' },
    isVerified: { type: Boolean, default: false },
    otp: { type: String },
    otpExpires: { type: Date },
  },
  { timestamps: true }
);

// Password hashing middleware
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) {
    next();
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

// Password comparison method
```

```
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

const User = mongoose.model('User', userSchema);
export default User;
```

Product Model

```
 import mongoose from 'mongoose';

const reviewSchema = mongoose.Schema(
  {
    user: { type: mongoose.Schema.Types.ObjectId, required: true, ref: 'User' },
    name: { type: String, required: true },
    rating: { type: Number, required: true },
    comment: { type: String, required: true },
  },
  { timestamps: true }
);

const productSchema = mongoose.Schema(
  {
    user: { type: mongoose.Schema.Types.ObjectId, required: true, ref: 'User' },
// Admin who added it
    name: { type: String, required: true },
    // FIX: Changed 'image' from a String to 'images' as an array of Strings.
    // This allows storing multiple image URLs for a single product.
    images: [{ type: String, required: true }],
    brand: { type: String, required: true },
```

```
    category: { type: String, required: true },
    description: { type: String, required: true },
    reviews: [reviewSchema],
    rating: { type: Number, required: true, default: 0 },
    numReviews: { type: Number, required: true, default: 0 },
    price: { type: Number, required: true, default: 0 },
    countInStock: { type: Number, required: true, default: 0 },
  },
  { timestamps: true }
);

const Product = mongoose.model('Product', productSchema);
export default Product;
```

Order Model

```
 import mongoose from 'mongoose';

const orderSchema = mongoose.Schema(
  {
    user: { type: mongoose.Schema.Types.ObjectId, required: true, ref: 'User' },
    orderItems: [
      {
        name: { type: String, required: true },
        qty: { type: Number, required: true },
        image: { type: String, required: true },
        price: { type: Number, required: true },
        product: { type: mongoose.Schema.Types.ObjectId, required: true, ref:
'Product' },
      },
    ],
    shippingAddress: {
      address: { type: String, required: true },
      city: { type: String, required: true },
```

```
      postalCode: { type: String, required: true },
      country: { type: String, required: true },
    },
    paymentMethod: { type: String, required: true },
    paymentResult: {
      id: { type: String },
      status: { type: String },
      update_time: { type: String },
      email_address: { type: String },
    },
    itemsPrice: { type: Number, required: true, default: 0.0 },
    taxPrice: { type: Number, required: true, default: 0.0 },
    shippingPrice: { type: Number, required: true, default: 0.0 },
    totalPrice: { type: Number, required: true, default: 0.0 },
    isPaid: { type: Boolean, required: true, default: false },
    paidAt: { type: Date },
    isDelivered: { type: Boolean, required: true, default: false },
    deliveredAt: { type: Date },
  },
  { timestamps: true }
);

const Order = mongoose.model('Order', orderSchema);
export default Order;
```

Repair Model

```
import mongoose from 'mongoose';

const repairSchema = mongoose.Schema(
  {
    customerName: { type: String, required: true },
    phoneNumber: { type: String, required: true },
    productName: { type: String, required: true },
    issueDescription: { type: String, required: true },
    status: {
      type: String,
      required: true,
      default: 'Pending',
      enum: ['Pending', 'In Progress', 'Resolved', 'Rejected'],
    },
  },
  { timestamps: true }
);

const Repair = mongoose.model('Repair', repairSchema);
export default Repair;
```

Warranty Model

```
 import mongoose from
'mongoose';


const warrantySchema =
mongoose.Schema(
  {
   user: { type:
mongoose.Schema.Types.ObjectId,
required: true, ref: 'User' },
   product: { type:
mongoose.Schema.Types.ObjectId,
required: true, ref: 'Product' },
   serialNumber: { type: String,
required: true, unique: true },
   purchaseDate: { type: Date,
required: true },
   warrantyPeriodInMonths: { type:
Number, required: true },
   status: {
     type: String,
     required: true,
     default: 'Claim Submitted',
     enum: ['Claim Submitted', 'In
Review', 'Approved', 'Rejected'],
   },
  },
  { timestamps: true }
);
```

```
const Warranty =
mongoose.model('Warranty',
warrantySchema);

export default Warranty;
```